

Random Walks

In this section, we'll use Python to generate data for a random walk and then use Matplotlib to create a visually appealing representation of that data. A *random walk* is a path that's determined by a series of simple decisions, each of which is left entirely to chance. You might imagine a random walk as the path a confused ant would take if it took every step in a random direction.

Random walks have practical applications in nature, physics, biology, chemistry, and economics. For example, a pollen grain floating on a drop of water moves across the surface of the water because it's constantly pushed around by water molecules. Molecular motion in a water drop is random, so the path a pollen grain traces on the surface is a random walk. The code we'll write next models many real-world situations.

Creating the RandomWalk Class

To create a random walk, we'll create a `RandomWalk` class, which will make random decisions about which direction the walk should take. The class needs three attributes: one variable to track the number of points in the walk, and two lists to store the *x*- and *y*-coordinates of each point in the walk.

We'll only need two methods for the `RandomWalk` class: the `__init__()` method and `fill_walk()`, which will calculate the points in the walk. Let's start with the `__init__()` method:

```
random ❶ from random import choice
_walk.py

class RandomWalk:
    """A class to generate random walks."""

    ❷ def __init__(self, num_points=5000):
        """Initialize attributes of a walk."""
        self.num_points = num_points

        # All walks start at (0, 0).
    ❸ self.x_values = [0]
        self.y_values = [0]
```

To make random decisions, we'll store possible moves in a list and use the `choice()` function (from the `random` module) to decide which move to make each time a step is taken ❶. We set the default number of points in a walk to 5000, which is large enough to generate some interesting patterns but small enough to generate walks quickly ❷. Then we make two lists to hold the *x*- and *y*-values, and we start each walk at the point (0, 0) ❸.

Choosing Directions

We'll use the `fill_walk()` method to determine the full sequence of points in the walk. Add this method to *random_walk.py*:

```
random
_walk.py

def fill_walk(self):
    """Calculate all the points in the walk."""
```
